

TriplePeek: From Entity Search to Live Linked Data Exploration

Albert Khomich^[0009-0006-9466-1783], Mohamed Ahmed
Sherif^[0000-0002-9927-2203], and Axel-Cyrille Ngonga
Ngomo^[0000-0001-7112-3516]

Data Science Group (DICE), Heinz Nixdorf Institute, Department of Computer
Science, Paderborn University
Paderborn, Germany
`akhomich@campus.uni-paderborn.de`

Abstract. Knowledge graphs expose structured information through SPARQL endpoints, yet finding an appropriate resource often requires knowledge of its identifier or the underlying vocabulary. We present TriplePeek, a configurable web interface that connects lexical entity discovery with live Linked Data exploration. A local PostgreSQL catalog stores resource identifiers, labels, and selected searchable metadata. Once a resource is selected, its IRI anchors an endpoint-provided description or an operator-defined SPARQL query. Query templates can delegate federated retrieval to the configured endpoint through SERVICE clauses. The interface combines navigable RDF tables and grouped query values with images and maps when suitable values are available. We describe the catalog lifecycle, ranking procedure, and query execution, alongside a bundled 10,000-resource Wikidata demonstration. Tests on 25 September 2026 covered DESCRIBE against six public and research-group endpoints, with additional checks of custom buttons and catalog generation. Search performance is evaluated separately from external endpoint execution. With a 17,795,730-entity catalog containing labels and aggregated types, seven visible browser-observed search requests took 256–1,370 ms (median 420 ms) on an Intel Xeon Silver 4310 server with 62 GiB RAM and mirrored SSD storage. These observations provide initial deployment evidence; controlled performance, retrieval-quality, and usability evaluation remain future work.

Keywords: Knowledge graphs · RDF · SPARQL · Entity search

1 Introduction

RDF identifies resources using IRIs and represents their relations as triples [3]. SPARQL provides expressive graph access [4], but users often begin with a name rather than an identifier or knowledge of the vocabulary. They need to find the corresponding resource before inspecting its graph context and the information available from other endpoints.

TriplePeek connects discovery and exploration through a shared IRI. Maintainers select searchable metadata for a local catalog and provide reusable SPARQL queries; users find an entity and open its actions without writing SPARQL. An independently maintained catalog can supply alternative labels, aggregated types, and task-specific terminology without modifying the source graph. This gives maintainers control over what users can discover, while endpoint-backed actions retrieve live information for the selected IRI. CSV files and SPARQL templates configure this exploration experience without requiring a custom frontend for each knowledge graph. The tradeoff is explicit catalog maintenance: discovery metadata may lag behind the live graph.

The system integrates (i) a catalog-to-IRI-to-query workflow, (ii) catalog generation and transactional import, and (iii) file-based query actions with RDF, media, and location views. This is an interface contribution, not a new indexing or federation algorithm. Source and documentation are available online.¹

Related work. YASGUI supports SPARQL editing and result visualization [8]; it serves users who formulate queries directly. TriplePeek instead offers lexical discovery followed by operator-defined actions, allowing visitors to explore without writing SPARQL. LodView renders navigable HTML views of RDF resources [2]; TriplePeek combines resource navigation with a separately maintained catalog that determines searchable terminology and coverage. SemFacet supports semantic faceted search and interactive query refinement [1]; TriplePeek uses a selected IRI across multiple predefined queries. The architectural contribution is the combination of configurable discovery metadata, reusable entity-anchored actions, and shared result views. An operator can adapt these components through CSV files and SPARQL templates while keeping the frontend unchanged. These capabilities describe a configuration and interaction model; they do not establish superiority over the cited systems.

2 Architecture and Data Flow

Figure 1 shows the architecture. The *search layer* maps text to records; the *application* displays candidates and instantiates actions; the *endpoint* executes queries and any configured federation. The implementation uses TypeScript, Next.js, React, PostgreSQL, N3.js, and MapLibre GL. Docker Compose separates the database, application, catalog generator, and importer.

Let E be the configured endpoint and C a finite catalog:

$$C = \{(u, \ell, \mathbf{m})\}, \quad q \mapsto C_q \mapsto u \mapsto \text{eval}_E(T[u]). \quad (1)$$

Here u is a resource IRI, ℓ its display label, $\mathbf{m}$ selected textual metadata, q a search string, and C_q the ranked candidates. A selected template T is instantiated with u . The built-in action uses `DESCRIBE <u>`. Its returned graph

¹ <https://github.com/dice-group/triple-peek>; documentation: <https://dice-group.github.io/triple-peek/>.

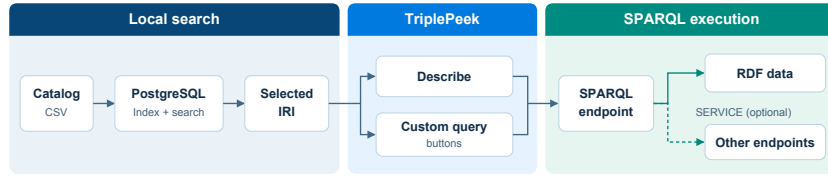


Fig. 1. TriplePeek workflow. Catalog search identifies an IRI; the application then requests a description or instantiates a query template. SERVICE clauses are executed by the configured endpoint. Responses may be RDF graphs, SELECT bindings, or ASK booleans. DESCRIBE returns an endpoint-defined resource description.

is determined by the endpoint; it is not guaranteed to contain every incident triple.

The catalog is a discovery projection, not an RDF replica. Metadata may combine graph values, alternative terminology, and curated descriptions; its coverage bounds text-search recall. Search requests access PostgreSQL only. Opening an action triggers SPARQL retrieval, so search input changes do not contact the endpoint. This separates workloads without establishing a latency improvement.

2.1 Catalog Preparation and Import

A CSV catalog contains at least `iri` and `label`; further columns become searchable metadata. Alternatively, the generator executes an operator-provided SELECT query against *E* and exports bindings using LIMIT/OFFSET pagination. Operators can request small pages and bound exported rows or UTF-8 file size. Query design must suit the endpoint’s vocabulary, data, and execution limits: richer metadata increases extraction work, while smaller pages reduce each response at the cost of more requests. Output limits bound catalog coverage. The generator writes complete CSV rows into a temporary file and replaces the destination only after successful export. Stable pagination requires a suitably ordered query and a sufficiently stable endpoint dataset; the bundled query does not enforce either condition.

Before importing, the pipeline normalizes the second column to `label` and validates headers, CSV structure, and IRIs. When the same IRI occurs in several rows, validation automatically groups the distinct, nonempty values in each field and retries validation. Thus, multiple type labels can be aggregated into one field alongside the resource label in a single catalog row per IRI. A PostgreSQL staging table holds validated rows. Schema changes, index creation, and insertion occur within one transaction, so an import failure leaves the search table unchanged. File normalization occurs separately and can modify the CSV.

By default, an IRI already present in the table causes insertion to fail. With `--update`, incoming rows replace searchable values for the corresponding IRIs; omitted metadata columns become NULL and absent IRIs remain unchanged.

The importer creates a full-text GIN index, a trigram GIN index, and an index on lower-case labels for prefix matching. All imported fields, including the IRI, contribute to the generated search text.

2.2 Entity Search and Ranking

Candidate retrieval combines exact labels, label prefixes, full-text matching, and trigram word similarity. Let x_c be the concatenated fields of a record c and v_c its PostgreSQL text-search vector. The priority of a matching record is the first satisfied condition:

$$p(c, q) = \begin{cases} 1 & \text{case-insensitive exact label,} \\ 2 & \text{case-insensitive literal label prefix,} \\ 3 & \text{full-text match against } v_c, \\ 4 & \text{trigram word-similarity match against } x_c. \end{cases} \quad (2)$$

Full-text matching uses `plainto_tsquery` with the `simple` configuration. Within priorities 1–3, the score is `ts_rank_cd`; within priority 4, it is `word_similarity` [5,6]. Trigram eligibility uses PostgreSQL’s configured word-similarity threshold. No learned model or embedding search participates in this ranking.

Results are ordered by ascending priority, descending score, lower-case label, original label, and IRI. A keyset cursor records this ordering tuple and the query string. The API returns at most 20 records per page and deduplicates by IRI before limiting. The interface waits 300 ms after an input change before requesting a page. Result cards show the label, IRI, and bounded metadata summaries; metadata too long to display can still contribute to search.

2.3 Query Actions and Presentation

Each regular `.sparql` file in the application’s buttons directory defines an action. Templates use the literal placeholder `${iri}` inside an IRI reference. For example, the following entity-summary action retrieves an image and coordinates:

```
PREFIX wdt: <http://www.wikidata.org/prop/direct/>
SELECT ?image ?coord WHERE {
  OPTIONAL { <${iri}> wdt:P18 ?image }
  OPTIONAL { <${iri}> wdt:P625 ?coord }
}
```

The server validates the supplied IRI, substitutes all placeholders, and recognizes `SELECT`, `ASK`, `CONSTRUCT`, or `DESCRIBE` after prefix/base declarations. Templates are trusted operator configuration; update forms are unsupported. Requests use the configured endpoint and a 15-second timeout. `SELECT` and `ASK` request SPARQL JSON results; graph queries prefer Turtle. A template

may include SERVICE clauses, whose execution and remote-source access are delegated to the endpoint [4].

Parseable RDF graph responses are displayed as subject–predicate–object tables, with a graph column where applicable. Selecting a named RDF term reruns the panel’s action for that IRI, and a local history supports backward navigation. SELECT results are grouped into distinct values per projected variable, and ASK results appear as booleans. Grouping suits entity attributes but does not preserve correlations between variables across result rows. Recognized image URLs and point coordinates add media previews and map markers; these views are heuristic interpretations of returned values.

3 Demonstration and Evidence

Scenario. The preloaded, hands-on demonstration uses the bundled Wikidata catalog of 10,000 distinct entities, with IRI, label, type label, and country fields. Its generation query selects castles with English labels, an image, and coordinates. Visitors complete four actions:

1. Search for *Neuschwanstein Castle* and select its IRI.
2. Open Describe and follow a named RDF term to another resource, then return to the castle using the panel history.
3. Open Details to display the castle’s image, English description, and mapped location.
4. Run Embedding to retrieve its precomputed Keci representation through a federated SERVICE query.

The fourth step retrieves an existing embedding; it does not compute embeddings or perform similarity search. A separate, operator-led fifth step replaces the catalog and a query template to show dataset maintainers how to adapt the same interface to another dataset. Visitors use the preloaded configuration for the hands-on steps. Live results depend on external services and may differ from catalog metadata.

Reproducibility paths. *Quick Start:* clone the repository and follow its documented Docker Compose setup using the bundled 10,000-entity catalog to reproduce the live demonstration. *Large-catalog experiment:* download the archived 17,795,730-entity CSV from Zenodo,² import it through the documented validation and PostgreSQL import pipeline, and run the local search workload reported below. Issue the seven searches below in the listed order with `limit=20`, with browser caching disabled. For live exploration of the large catalog, replace the default `SPARQL_ENDPOINT` with `https://dbpedia.data.dice-research.org/sparql` and use DBpedia-compatible query templates. This Tentrism deployment serves the full DBpedia 2022-12 snapshot used for this setup. The official DBpedia endpoint has restricted dataset coverage and query limits,³ so it is not the endpoint

² <https://zenodo.org/records/22963871>.

³ <https://www.dbpedia.org/resources/sparql/>.

Table 1. Endpoints tested with DESCRIBE on 25 September 2026. DICE snapshot and Tentrism version information was supplied by the authors. Linked service documentation identifies the public-service backends; their tested versions were not supplied.

Endpoint	Dataset or snapshot	Backend
https://sparql.embeddings.cc/sparql	DBpedia and Wikidata embeddings	em- Tentrism 1.0.0
https://dbpedia.data.dice-research.org/sparql	DBpedia snapshot 2022-12	Tentrism 1.0.0
https://wikidata.data.dice-research.org/sparql	Wikidata 2025-01-22 beta	truthy Tentrism 1.0.0
https://query.wikidata.org/sparql	Official Wikidata Query Service	Blazegraph
https://dbpedia.org/sparql	Official DBpedia public endpoint	Virtuoso
https://semrepo.org/sparql	GitHub repositories linked to research	Virtuoso

used for this path. This setting affects live actions, not local search timing. The two paths reproduce different components: the live visitor workflow and local search over a loaded large catalog. Neither requires timing external SPARQL services.

Experimental environment and scope. Experiments were conducted on 25 September 2026 on a server equipped with an Intel Xeon Silver 4310 processor (12 physical cores, 24 hardware threads, 2.10 GHz) and 62 GiB RAM. PostgreSQL data resided on two Samsung 240 GB SSDs configured in RAID 1 with the ext4 filesystem. The application and database were deployed using Docker Compose. We assess local catalog search and functional compatibility of query buttons and catalog generation. External endpoint response times are outside this evaluation. Search accesses PostgreSQL independently of the configured endpoint; changing that endpoint with the catalog and search configuration fixed does not require repeating the search measurements.

Endpoint interoperability. DESCRIBE succeeded against all six endpoints in Table 1, covering embeddings, general knowledge, and research software. Sem-Repo links repositories to scholarly context [7]. Selected custom buttons and the catalog generator were also tested across endpoints; a complete endpoint-by-action coverage matrix was not recorded. Button applicability depends on the query and available data, while generator queries and page sizes require endpoint-specific configuration. Fuseki support was additionally checked using a temporary test endpoint.

Large-catalog observations. The loaded catalog held 17,795,730 entities, one row per entity with labels and aggregated types. A browser Network-panel capture shows seven visible HTTP 200 search requests with `limit=20: dublin` (707 ms),

`irlnad` (307 ms), `irland` (1.37 s), `paderborn` (305 ms), `hein nixdor` (636 ms), `hein nixdorf` (420 ms), and `henin nixdorf` (256 ms). Query spellings are preserved from the capture; URL-encoded plus signs denote spaces. The seven displayed durations range from 256 to 1,370 ms, with a median of 420 ms. Although the panel footer lists 17 requests, only the seven visible search rows are analyzed. These HTTP durations characterize the search service with the large catalog loaded; they exclude the pre-request debounce and include the browser–application exchange rather than isolating SQL execution. Search does not invoke SPARQL endpoints. The browser cache was disabled; this does not imply cold database or operating-system caches. The capture provides observed request latencies, not a throughput measurement or a latency distribution under concurrent load.

Conference deployment. The proposed conference deployment runs the application and PostgreSQL on a laptop with the demonstration catalog already imported. Visitors perform the four hands-on actions above; the presenter then leads the configuration walkthrough. Internet access is needed for live queries and external media. A recorded walkthrough and captured example responses will be prepared as an explicitly labeled fallback for service outages.

Evidence and evaluation scope. The bundled regression suite checks query-file discovery, supported query forms, IRI substitution, typed responses, and error handling using mocked endpoint responses. All eight scenario subtests passed in a local execution. These automated tests complement the exploratory endpoint and large-catalog checks above; they do not measure ranking quality or user experience. Further evaluation should measure local search latency and throughput under a controlled workload, retrieval quality with relevance judgments, generator completion and output validity, button compatibility by endpoint, and task completion with representative users.

4 Limitations and Ethical Use of Data

Catalog updates are explicit; no synchronization mechanism guarantees agreement with live RDF. Recall is bounded by the chosen resources and terms. Duplicate grouping is performed in memory, so streaming validation does not imply bounded memory for the whole pipeline. The primary endpoint must support any SERVICE clauses in the templates. Large responses are processed in memory, SELECT displays discard tuple associations, and the interactive RDF parser does not handle JSON-LD or RDF/XML. These constraints motivate bounded, entity-summary queries and further interoperability testing.

The demonstration uses public Wikidata metadata, and the exploratory tests additionally used DBpedia, SemRepo, and DICE-hosted datasets. No human-participant study or personal search-log collection is reported. Deployments must consider data and image licenses, provenance, endpoint policies, and the suitability of external services. Future user studies should obtain appropriate consent and minimize retained interaction data.

5 Conclusion

TriplePeek connects a curated lexical catalog to live, IRI-anchored SPARQL exploration. Its configurable metadata and query actions let maintainers adapt the discovery workflow while leaving RDF execution with existing endpoints. Exploratory use across several endpoint implementations and a 17,795,730-entity catalog provides initial evidence of practical applicability. Controlled retrieval, performance, and usability evaluation are the next steps.

References

1. Arenas, M., Grau, B.C., Kharlamov, E., Marciuška, Šarūnas., Zheleznyakov, D., Jimenez-Ruiz, E.: SemFacet: Semantic faceted search over Yago. In: Proceedings of the 23rd International Conference on World Wide Web (Companion). pp. 123–126. Association for Computing Machinery, New York, NY, USA (2014). <https://doi.org/10.1145/2567948.2577011>, <https://www.cs.ox.ac.uk/isg/tools/SemFacet/SemFacetwww14.pdf>
2. Camarda, D.V., Antonuccio, A.: LodView: IRI dereferencer, RDF to HTML. Software repository, <https://github.com/LodLive/LodView>, accessed 24 September 2026
3. Cyganiak, R., Wood, D., Lanthaler, M.: RDF 1.1 concepts and abstract syntax. W3c recommendation, World Wide Web Consortium (Feb 2014), <https://www.w3.org/TR/2014/REC-rdf11-concepts-20140225/>, editors; 25 February 2014
4. Harris, S., Seaborne, A.: SPARQL 1.1 query language. W3c recommendation, World Wide Web Consortium (Mar 2013), <https://www.w3.org/TR/2013/REC-sparql11-query-20130321/>, editors; 21 March 2013
5. PostgreSQL Global Development Group: PostgreSQL 16 Documentation: Controlling Text Search, <https://www.postgresql.org/docs/16/textsearch-controls.html>, accessed 24 September 2026
6. PostgreSQL Global Development Group: PostgreSQL 16 Documentation: pg_trgm, <https://www.postgresql.org/docs/16/pgtrgm.html>, accessed 24 September 2026
7. Rafay, A., Susanti, Y., Lamprecht, D., Färber, M.: SemRepo: A knowledge graph for research software and its scholarly ecosystem (2026). <https://doi.org/10.48550/arXiv.2605.13310>, <https://arxiv.org/abs/2605.13310>
8. Rietveld, L., Hoekstra, R.: The YASGUI family of SPARQL clients. *Semantic Web* 8(3), 373–383 (2017). <https://doi.org/10.3233/SW-150197>, <https://doi.org/10.3233/SW-150197>